

A Graph-Based Deep Reinforcement Learning Framework for Quantum Circuit Mapping with Look-Ahead Rewards and Biased Exploration

Yueran Zhao^{1,2}, Kaiqi Li¹, Ziyang Guo¹, Jialin Zhang^{1,2}

¹State Key Lab of Processors, Institute of Computing Technology
Chinese Academy of Sciences, Beijing 100190, China

²School of Computer Science and Technology
University of Chinese Academy of Sciences, Beijing 100049, China

Abstract—Current quantum computing hardware imposes strict constraints on two-qubit gate operations, limiting their application to specific qubit pairs. This leads to a structural mismatch between quantum circuits and hardware limitations. Quantum circuit mapping addresses this issue by inserting SWAP gates to enable logical circuit equivalence. In the Noisy Intermediate-Scale Quantum (NISQ) era, minimizing the number of additional gates is crucial to reduce circuit error rates. We propose LARBE (Look-Ahead Rewards and Biased Exploration), a deep reinforcement learning framework utilizing graph neural networks to encode quantum circuits. The look-ahead reward mechanism enables intelligent trade-offs between immediate and future mapping benefits, while the biased exploration incorporates heuristic rules to accelerate training convergence. Benchmarking against the SABRE algorithm demonstrates performance improvements ranging from 5.83% to 17.57% on several common hardware topologies.

Index Terms—Quantum computing, Quantum circuit mapping, Reinforcement Learning

I. INTRODUCTION

Quantum computing algorithms offer significant theoretical advantages for solving problems such as integer factoring [1] and unstructured search [2], demonstrating potential to address challenges intractable for classical computers. Beyond these, quantum computing has shown promise in diverse fields, including chemistry [3], combinatorial optimization [4], and machine learning [5]–[8]. Over the past decades, quantum hardware has advanced significantly, enabling researchers to execute quantum algorithms on devices with around 100 qubits, albeit in noisy environments. This progress brings us closer to large-scale, fault-tolerant quantum computing. However, current quantum devices, operating in the Noisy Intermediate-Scale Quantum (NISQ) era [9], [10], face limitations such as short qubit coherence times, low gate fidelity, and restricted connectivity. These constraints necessitate the transpilation of theoretical quantum circuits into hardware-compatible versions, a process known as quantum circuit mapping or qubit routing. The goal of quantum mapping is to transform input circuits into equivalent forms that satisfy hardware constraints while minimizing overhead, particularly for two-qubit gates.

The qubit mapping problem is a critical component of quantum circuit compilation and has attracted extensive research efforts. Numerous mapping algorithms have been proposed [11]–[15], aiming to improve the execution fidelity of quantum circuits. However, finding the optimal mapping strategy is proven to be NP-Hard [16], [17], highlighting the inherent complexity of the problem. Current solutions can be broadly categorized into two approaches. The first approach formulates the mapping problem as a mathematical

optimization task, leveraging solvers for exact or approximate solutions [13], [18]–[22]. For instance, Shafaei et al. [18] transformed the mapping problem on one-dimensional and lattice topologies into minimum linear arrangement and mixed-integer programming problems, respectively. While effective for small-scale circuits, this approach faces scalability challenges. The second approach employs heuristic algorithms, using cost functions to evaluate and insert SWAP gates [14], [15], [18], [23]–[26]. For example, Zulehner et al. [26] proposed an A*-based algorithm that partitions circuits into layers and optimizes gate placement hierarchically. Similarly, Li et al. [14] introduced SABRE, a heuristic algorithm that uses distance-based cost functions to greedily select SWAP gates, enabling efficient mapping for large-scale circuits. While heuristic methods are computationally efficient, they often lack theoretical guarantees, leaving room for further optimization.

Recent advancements in artificial intelligence, particularly in machine learning, have opened new avenues for addressing the quantum bit mapping problem. Prior work, such as the noise-adaptive quantum circuit mapping method using reinforcement learning and graph neural networks [27], as well as reinforcement learning-based qubit routing techniques [28], has demonstrated the potential of data-driven approaches in this domain. While these studies have laid a foundation for leveraging machine learning in quantum compilation, further improvements in training efficiency and mapping quality remain critical challenges.

In this study, we propose LARBE, a deep reinforcement learning framework based on graph convolutional networks (GCNs), designed to optimize quantum circuit mapping by accelerating training convergence and enhancing solution quality. Our method employs GCNs to encode quantum circuit states and introduces a tailored reinforcement learning framework that incorporates look-ahead rewards and biased exploration for more effective decision-making. We train the model across diverse hardware topologies and further extend the algorithm to generalize to larger-scale architectures. Compared to the baseline SABRE algorithm, LARBE achieves superior performance in mapping random quantum circuits, validating its effectiveness in both efficiency and optimization.

The contributions of this work are summarized as follows:

- We propose a graph-based deep reinforcement learning framework for quantum circuit mapping, combining graph convolutional neural networks (GCNs) with a Deep Q-Network (DQN) architecture. This framework efficiently encodes quantum circuits as graph structures and leverages the DQN to determine optimal SWAP gate insertion points, enabling effective mapping across diverse hardware topologies.

Supported in part by the National Natural Science Foundation of China Grants No. 92465202 and No. 62272441.

- We introduce a sliding block partitioning scheme for quantum circuits, ensuring uniform training across circuits of varying sizes. This method effectively utilizes the internal structural information of quantum circuits, enhancing the learning process and scalability of the algorithm.
- We enhance the reinforcement learning framework with look-ahead rewards and biased exploration strategies. The look-ahead rewards encourage actions that benefit future mappings, while the biased exploration strategy accelerates training by prioritizing actions with higher potential for improving circuit mapping.
- We validate our approach through extensive experiments on multiple hardware topologies, demonstrating its effectiveness in reducing SWAP gate insertions and improving mapping efficiency across diverse quantum computing architectures.

II. PRELIMINARIES

A. Quantum Circuits

In the gate model of quantum computing, quantum circuits are composed of sequences of reversible gates. Quantum gates manipulate logical qubits, transforming their quantum states. The state of a qubit is represented as a linear superposition of basis states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix},$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. Quantum gates act on qubits through unitary transformations, which can be represented by unitary matrices. In the context of quantum bit mapping, the most frequently used gate is the SWAP gate, represented by a 4×4 unitary matrix defined as follows:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The SWAP gate operates on two qubits, exchanging their quantum states. For example, it transforms the state $|i\rangle|j\rangle$ into $|j\rangle|i\rangle$.

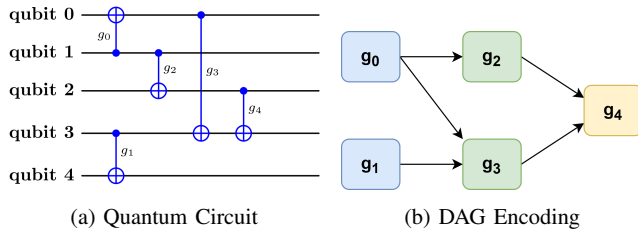


Fig. 1: An example of encoding a quantum circuit into a directed acyclic graph (DAG).

In a quantum circuit, gates can be executed in parallel if they do not share common qubit targets. However, gates with shared qubit targets or implicit execution orders must be executed sequentially. To represent such execution dependencies, a directed acyclic graph (DAG) $G = (V, E)$ is constructed. Each vertex $v_g \in V$ corresponds to a gate g in the circuit, and a directed edge $v_{g_1} \rightarrow v_{g_2}$ indicates that gate g_1 must be executed before gate g_2 . Each gate is assigned a unique ID, which serves as a vertex label in the DAG. This labeling ensures a one-to-one correspondence between the DAG and the quantum circuit, independent of gate types. Examples of a quantum circuit and its corresponding DAG encoding are shown in Figure 1a and Figure 1b, respectively.

B. Quantum Circuit Mapping Problem

The quantum circuit compilation problem focuses on translating logical quantum operations into executable circuits compliant with a quantum processor's topological constraints. A central challenge is qubit mapping, where logical-to-physical qubit assignments must align with limited hardware connectivity. The SWAP gate addresses this by exchanging qubit states to enable required two-qubit interactions, as illustrated in Figure 2. However, excessive SWAP insertions degrade circuit fidelity through increased depth and error rates. Optimal strategies must therefore balance SWAP minimization, gate count reduction, and fidelity preservation that is the key challenges in scalable quantum computing.

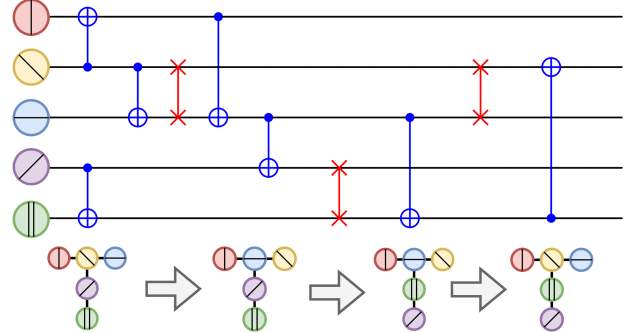


Fig. 2: Temporal dynamics of logical-to-physical qubit mapping relations

C. Graph Neural Network

Graph neural network [29] is a kind of neural network that deals with graph data, extracting the structure and information with a neural network representation. As an analogy to convolutional neural networks developed in computer vision domains, convolutional graph neural networks capture the information of a vertex by repeatedly absorbing features of its neighbors. The method of graph convolution is essential to build large and sophisticated graph neural networks. Many convolutional graph neural network variants, such as GCN, GAT and SAGE, share a similar design. Denote the feature vector of vertex i after l -th convolution is h_i^{l+1} . It is retrieved by combining features of neighbors $N(i)$ with a learnable kernel W^l

$$h_i^{l+1} = \sigma \left(\sum_{j \in N(i)} \frac{1}{c_{ij}} W^l h_j^l \right)$$

where σ is an activation function and c_{ij} some normalization factor. Various modifications may be presented as network performance improvement. The graph convolutional layer technology used in this article comes from GraphSage proposed by Hamilton et al [30].

D. Deep Q-Learning

Q-learning [31] is a reinforcement learning algorithm designed to maximize the expected reward of an agent interacting with an environment. Let S denote the set of agent states and A the action space. At time step t , if the agent is in state $s \in S$ and takes action $a \in A$, it receives a reward r_t from the environment. The total return from time t is defined as the discounted sum of future rewards:

$$R_t = \sum_{k=t}^T \gamma^{k-t} r_k,$$

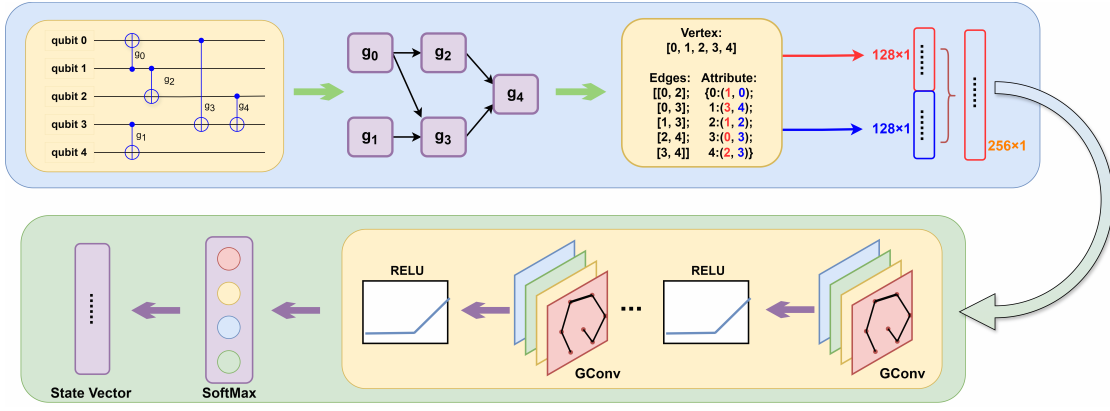


Fig. 3: The architecture of the circuit encoding network.

where T is the terminal time step and $\gamma \in [0, 1]$ is the discount factor.

The relationship between Q values of the current state and the next state is described by the Bellman equation:

$$Q^*(s, a) = r + \gamma \max_{a'} Q^*(s', a'),$$

where s' is the next state after taking action a' in state s .

In the context of quantum circuit mapping, the action space A_L consists of all edges in the hardware layout L that support two-qubit gates. The agent's state corresponds to the quantum circuit or its equivalent representation. To approximate the optimal $Q^*(s, a)$, we train a neural network $Q(s, a; \theta)$ [32].

III. LARBE: AN END-TO-END APPROACH

We propose LARBE, a novel framework that combines graph learning and reinforcement learning techniques to address the quantum circuit mapping problem. Given a hardware layout L , LARBE takes a quantum circuit as input and iteratively selects optimal SWAP gate positions to adapt the circuit to the hardware constraints.

In this section, we detail the key components of LARBE, including circuit encoding techniques, circuit segmentation rules, the reinforcement learning network architecture, and the Q-learning training procedure. These components collectively enable the training of an intelligent agent capable of solving the circuit mapping problem. Additionally, we introduce an expansion algorithm that generalizes the model trained on small-scale topologies to larger-scale topologies with similar structures.

A. Circuit Encoding Network

To encode the quantum circuit, we first transform it into a directed acyclic graph (DAG) enriched with additional information. Each node in the DAG represents a two-qubit gate in the logical circuit and is annotated with a pair of integers (q_i, q_j) indicating the two qubits acted upon by the gate.

The encoding process employs a single shared embedding table to project all qubit indices (both q_i and q_j) into a unified 128-dimensional space. For each node representing a two-qubit gate, the corresponding q_i and q_j embeddings are vertically concatenated, forming a 256-dimensional node representation that preserves the directional relationship between control and target qubits. These composite vectors are then processed through multiple layers of GraphSage convolutional layers interleaved with LeakyReLU activation layers. The GraphSage architecture is chosen for its ability

to efficiently aggregate neighborhood information while preserving node identity, which is critical for maintaining gate dependency relationships. LeakyReLU activations help alleviate gradient vanishing issues during backpropagation.

Through successive graph convolutions and activations, the information of each node is fully propagated across the graph. To ensure numerical stability and accelerate training, we apply a graph normalization layer [33], which normalizes the node vectors without disrupting the graph's structural information. Finally, a Softmax aggregation layer [34] condenses the information from all nodes into a single compact vector representation, where gates with stronger dependencies automatically receive higher attention weights. This hierarchical encoding scheme effectively captures both local gate operations and global circuit structures, as visualized in Figure 3.

B. Circuit Segmentation

Handling large-scale quantum circuits poses significant challenges for the proposed neural network due to computational and memory constraints. To address this, we introduce a gate size threshold M_g for circuit segmentation. This threshold ensures that the input to the neural network remains manageable by breaking down large circuits into smaller segments, each containing at most M_g gates. The choice of M_g is carefully calibrated to balance computational efficiency and the preservation of circuit structure.

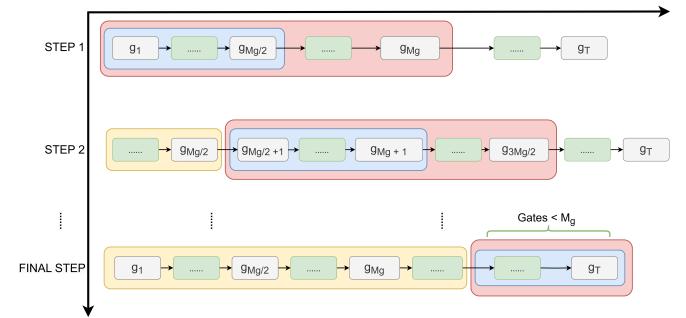


Fig. 4: An example of the sliding window segmentation strategy for large-scale quantum circuits.

To maximize the utilization of the circuit's internal structure, we employ a sliding window segmentation strategy as illustrated in Figure 4. The process begins by extracting a segment of M_g gates (marked by the red box) and mapping approximately the first half (highlighted in blue), while reinserting the remaining gates into

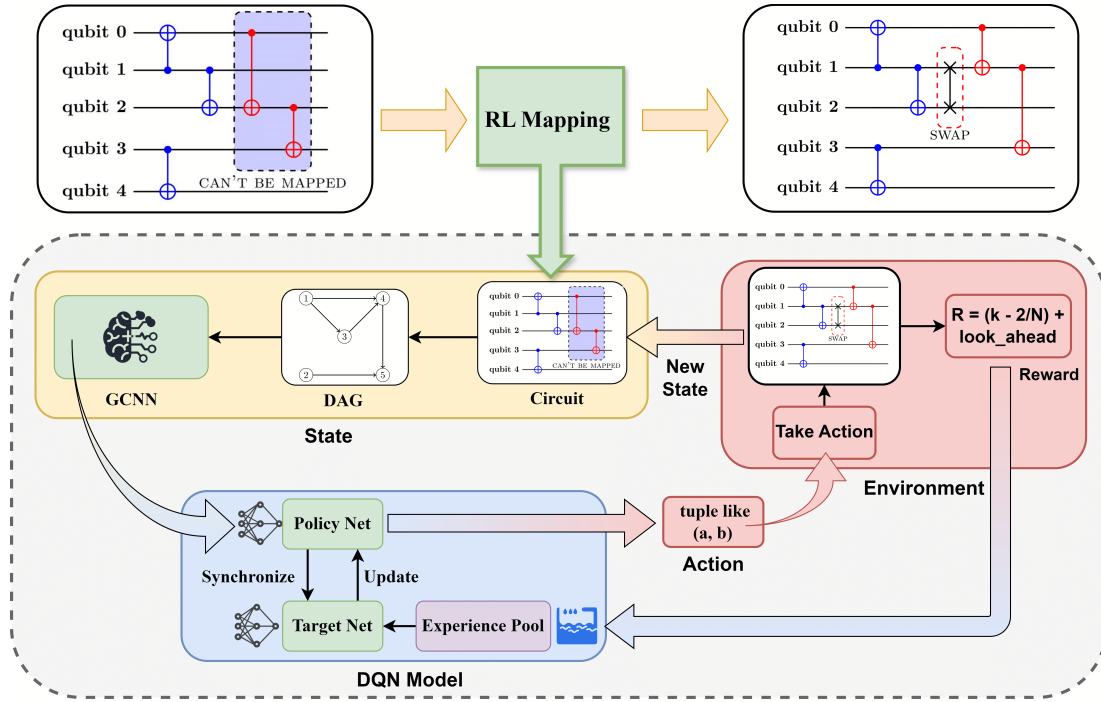


Fig. 5: The reinforcement learning architecture for quantum circuit mapping. The agent's state is encoded by the graph encoding network, which transforms the quantum circuit into a high-dimensional vector representation. The agent's policy is determined by a Deep Q-Learning network, consisting of a policy network and a target network. The environment interacts with the agent by generating new states and rewards based on the agent's actions, enabling iterative optimization of the mapping process.

the processing queue. The window slides forward iteratively in this manner until meeting the termination condition where the number of unmapped gates becomes strictly less than M_g , at which point all residual gates are mapped in one final batch. Throughout this process, the yellow box serves as a visual record of all successfully mapped circuit portions. This approach guarantees full circuit coverage while maintaining structural dependencies through carefully designed segment overlaps.

The sliding segmentation strategy offers several advantages. By dynamically adjusting the segmentation window during the quantum circuit mapping process, it effectively addresses two critical challenges: (1) preserving structural information at segment boundaries, and (2) minimizing the computational overhead of dynamically constructing circuit segments. This approach ensures efficient handling of large-scale circuits while maintaining the integrity of the circuit structure.

Since the segmentation of large circuits and the mapping of individual sub-circuits are independent processes, we focus in subsequent sections on a single sub-circuit, i.e., a segment containing no more than M_g quantum gates. This simplification allows us to concentrate on optimizing the mapping process within manageable segments, thereby enhancing the algorithm's efficiency and scalability.

C. Reinforcement Learning Architecture

Figure 5 illustrates the overall architecture of our reinforcement learning (RL) framework for quantum bit mapping. The quantum circuit is encoded into the agent's state, which is processed by a decision network to select an action. The action corresponds to inserting a SWAP gate into the circuit, after which the environment updates the circuit state and provides a reward. This iterative process

continues until the circuit is fully mapped to the hardware. Below, we detail the key components of this framework.

1) *Circuit State Representation*: Directly representing a quantum circuit as a set of qubit lines and gates is unsuitable for RL. Instead, we convert the circuit into a directed acyclic graph (DAG) based on gate execution order, as described in Section II. This DAG captures the circuit's structural information and is encoded into a high-dimensional vector using graph convolutional networks (GCNs). This vector serves as the state representation in the RL framework. The encoding ensures that circuits with similar structures yield similar vectors, while distinct circuits produce different vectors. However, the infinite space of quantum circuits necessitates the use of deep reinforcement learning to handle the unbounded state space.

2) *Action Space*: In the context of quantum bit mapping, an action corresponds to inserting a SWAP gate at a specific location on the hardware topology. The action space is defined by the set of edges in the hardware's physical topology graph, as SWAP gates can only be inserted on these edges. Thus, the action space is finite and determined by the hardware constraints.

3) *Reward Design*: The reward function is meticulously designed to address two fundamental challenges in quantum circuit mapping: the sparsity of meaningful feedback signals and the necessity for multi-step strategic planning. By integrating both immediate operational feedback and long-term mapping potential assessment, this design guides the agent toward globally optimal SWAP insertion strategies while maintaining computational efficiency.

The immediate reward component $(k - \frac{2}{n}) \cdot C_R$ serves as the foundation for local optimization. Here, k directly reflects the action's effectiveness in resolving current mapping constraints - each successfully enabled two-qubit gate contributes linearly to the reward signal. The subtractive term $\frac{2C_R}{n}$ implements a topology-aware regulariza-

tion mechanism derived from quantum routing dynamics. In typical n -qubit architectures, the average distance between randomly selected qubit pairs follows an $O(n/2)$ scaling relationship. This statistical property ensures the penalty term automatically adapts to different hardware scales: larger quantum processors impose proportionally smaller penalties per SWAP operation, while smaller devices maintain stricter control over redundant actions. Specifically, when $k = 1$, the agent must complete the gate mapping within $\lceil n/2 \rceil$ SWAP operations to achieve positive cumulative rewards, creating a self-regulating mechanism against inefficient routing patterns.

The look-ahead reward term $\frac{M}{M_g} C_M$ introduces a critical innovation in temporal credit assignment. The numerator M evaluates the action's cascading effects by counting all gates that become executable in the current circuit segment after the SWAP insertion. This comprehensive assessment captures both direct and indirect benefits of qubit repositioning - even when no gates are immediately enabled ($k = 0$), actions that resolve multiple future routing bottlenecks receive proportional rewards. The denominator M_g (segment size) serves as a normalization factor that maintains reward magnitude consistency across different circuit partitioning schemes. Through this design, the agent learns to optimize qubit layouts not just for individual gates, but for entire sequences of operations within each segment.

The complete reward formulation:

$$R(C, a) = \left(k - \frac{2}{n}\right) \cdot C_R + \frac{M}{M_g} \cdot C_M,$$

where:

- k : Gates immediately executable after the action
- n : Total qubits in the circuit
- C_R : Scaling factor for reward stability
- M_g : Current circuit segment size
- M : Executable gates in the entire segment
- C_M : Scaling factor for future rewards

carefully balances immediate actions and long-term goals. The hyperparameters C_R and C_M control this fundamental trade-off: C_R determines the urgency of resolving immediate mapping constraints, while C_M regulates the planning horizon depth. Through empirical analysis, we establish that fixed parameter values yield robust performance across diverse hardware topologies, demonstrating the architecture's inherent stability. This stability originates from two key normalization mechanisms: the $\frac{1}{n}$ scaling in the penalty term automatically adapts to qubit count variations, while the $\frac{M}{M_g}$ ratio ensures consistent reward magnitudes across different circuit complexities. The design deliberately avoids extensive parameter tuning, prioritizing generalizability over marginal accuracy improvements - a crucial feature for practical quantum compilation workflows.

4) *Environment Augmentation*: Each quantum circuit to be mapped onto physical hardware constitutes a distinct reinforcement learning environment. However, circuits vary significantly in structure, and an agent must explore each environment extensively to maximize long-term rewards. Traditionally, new circuits are introduced only after the current circuit is fully executed or the training epoch ends. This approach can lead to overfitting in the early stages of training, as the agent's experiences become highly correlated, even with the use of replay memory in the DQN algorithm.

To mitigate this issue, we propose an environment rotation strategy. Multiple circuits (environments) are generated for each training epoch, with all context information preserved. During training, the agent randomly selects one of these environments at each exploration step. This "context switching" technique enhances the neural

network's ability to generalize experiences from diverse circuits, improving overall performance.

D. Training Steps

At the start of each training epoch, N_{env} random circuits with at most g_{max} gates are generated as rotating environments. During the epoch, the agent performs N_{ex} explorations, each involving the random selection of a circuit C from the environments. The agent employs a fused strategy to choose an action, combining the current policy network with uniformly random selection and a biased random selection guided by a greedy algorithm.

To balance exploration and exploitation, we introduce a progressive parameter $\epsilon(t) = p_1 + p_2 \cdot e^{-t/D}$, where t is the global time step, D is a decay factor, and $p_1 \approx 0$, $p_2 \approx 1$ are hyperparameters. The action selection strategy is as follows:

- With probability $\epsilon(t)/2$, the agent selects a legal SWAP position uniformly at random from A_L .
- With probability $\epsilon(t)/2$, the agent samples from a biased distribution. The probability of selecting action a is weighted by Δ_a , the decrease in the total distance of 2-qubit gate targets after applying a :

$$p(a) = \frac{\text{scale}(\Delta_a)}{\sum_a \text{scale}(\Delta_a)},$$

where $\text{scale}(x)$ is a modified ReLU with a small shift $\delta \ll 1$ at zero:

$$\text{scale}(x) = \begin{cases} x, & x > 0 \\ \delta, & x = 0 \\ 0, & x < 0. \end{cases}$$

- With probability $1 - \epsilon(t)$, the agent selects the best SWAP action according to the policy network.

The action at step t is thus selected as:

$$a_t = \begin{cases} \arg \max_a \pi(C, a), & \text{with probability } 1 - \epsilon(t), \\ \text{Sample}(A_L), & \text{with probability } \epsilon(t)/2, \\ \text{BiasedSample}(A_L), & \text{with probability } \epsilon(t)/2. \end{cases}$$

The reward for action a_t is calculated as:

$$R(C, a_t) = \left(k - \frac{2}{n}\right) \cdot C_R + \frac{M}{M_g} \cdot C_M,$$

where k is the number of gates executable due to a_t . After applying a_t , the resulting circuit C' and the tuple $(C, C', a_t, R(C, a_t))$ are stored in replay memory.

For training, a minibatch of size N_{batch} is sampled from replay memory. Each entry is a tuple $(C^{(i)}, C'^{(i)}, a^{(i)}, R(a^{(i)}))$. The expected reward is:

$$\hat{V}(C^{(i)}) = V(C'^{(i)}) + \gamma \cdot R(C^{(i)}, a^{(i)}).$$

The loss is computed using a smoothed L_1 loss over the minibatch:

$$\sum_i \text{SmoothL}_1 \left(V(C^{(i)}), V(C'^{(i)}) + \gamma \cdot R(C^{(i)}, a^{(i)}) \right).$$

A gradient step is then applied to minimize this loss.

E. Expansion Algorithm

In our reinforcement learning framework, each trained model is specific to a particular quantum hardware topology. To leverage models trained on smaller topologies for larger ones, we propose an expansion algorithm. This approach is particularly effective for grid topologies, where a larger grid (e.g., 5×5) can be decomposed into smaller, overlapping subgrids (e.g., 3×3) that match the topology of the trained model. By concentrating quantum gates within these subgrids and applying the trained model iteratively, we can efficiently map large circuits. Algorithm 1 details this process.

Algorithm 1 Mapping Expansion Algorithm

```

1: Input: Circuit  $C$ , Trained RL Model  $M$ , Initial Mapping  $\pi$ 
2: Output: Final Mapping  $\pi_f$ , Mapped Circuit  $mapped\_circ$ 
3:  $l \leftarrow \text{depth of } C$ 
4:  $mapped\_circ \leftarrow \emptyset$ 
5: for  $i = 0$  to  $l - 1$  do
6:    $gate\_list \leftarrow \text{Gates at Depth } i$ 
7:   while  $gate\_list \neq \emptyset$  do
8:     for each  $gate$  in  $gate\_list$  do
9:       Assign  $gate$  to a subgrid based on  $\pi$ 
10:    end for
11:    Select the subgrid with the most gates
12:     $gates \leftarrow \text{Gates in selected subgrid}$ 
13:    Map  $gates$  using model  $M$ 
14:     $gate\_list \leftarrow gate\_list \setminus gates$ 
15:     $mapped\_circ.add(gates)$ 
16:    Update  $\pi$  based on the new mapping
17:  end while
18: end for
19: return  $\pi_f$ ,  $mapped\_circ$ 

```

The algorithm processes the circuit layer by layer, ensuring parallel execution of gates within each layer. For each layer, gates are grouped into subgrids based on the initial mapping π . The subgrid with the most gates is selected and mapped using the trained model M . The mapping relationship π is updated after each subgrid is processed, and the algorithm iterates until all gates in the layer are mapped. This process is repeated for all layers, resulting in a fully mapped circuit.

IV. EXPERIMENTS

In this section, we evaluate the performance of our proposed algorithm against the widely-used SABRE algorithm. While SABRE is topology-agnostic, our approach leverages hardware topology to train reinforcement learning models. To ensure a fair comparison, we test both algorithms on multiple hardware topologies. The experiments are organized as follows:

- Section IV-A provides background information, including hardware topologies, test circuits, and experimental setup.
- Section IV-B presents the main results, comparing our algorithm with SABRE across various topologies.
- Section IV-C discusses the ablation study results.
- Section IV-D evaluates the performance of the expansion algorithm introduced in Section III-E.

A. Preliminary Information

The hardware topologies used in our experiments are illustrated in Figure 6. We generated random test circuits with sizes ranging from 200 to 1000 two-qubit gates. The experiments were conducted on an Ubuntu 24.04 system with an NVIDIA GeForce RTX 3090 GPU and

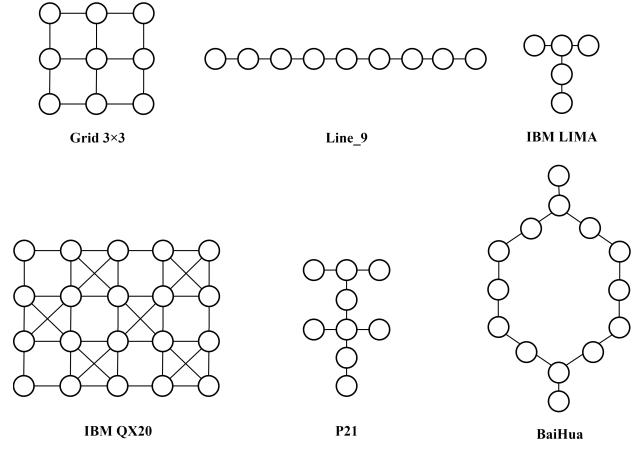


Fig. 6: Hardware topologies used in the experiments.

CUDA 12.4. For comparison, we used the SABRE-decay algorithm implemented in IBM's Qiskit framework.

The hyper-parameters for model training are listed in Table I.

Hyperparameter	Value
Replay pool size	10,000,000
Environment rotation	5
Batch size	160
Learning rate	0.00002
Gamma	0.95
Total epochs	2000
Explore period	10000
Target update period	10
Epsilon start	0.95
Epsilon end	0.05

TABLE I: Hyperparameter values used in model training.

B. Comparison with SABRE

SABRE, a heuristic search-based quantum circuit mapping algorithm, is widely adopted as the default mapping algorithm in IBM's Qiskit framework. To comprehensively evaluate the performance of our proposed algorithm, we conduct a comparative study against SABRE across multiple hardware topologies. The test circuits used in this study are randomly generated, with sizes ranging from 200 to 1000 two-qubit gates. The results are visualized in Figure 7 and summarized in Table II.

Topology	Total Gates	g_{rl}	g_{sabre}	Ratio
Grid 3x3	73703	36014	39974	9.94%
Line_9	72076	89262	103703	13.35%
IBM LIMA	71708	31113	37748	17.57%
IBM QX20	68174	47014	49922	5.83%
P21	70539	55874	64479	13.33%
BaiHua	71142	86153	91685	6.04%

TABLE II: Comparison of inserted SWAP gates between the proposed algorithm (g_{rl}) and SABRE (g_{sabre}) across different topologies. The total number of gates in the test circuits is also provided for reference.

The performance improvement ratio is defined as:

$$\text{Ratio} = \frac{g_{sabre} - g_{rl}}{g_{sabre}} \times 100\%.$$

From the data in Table II, the proposed algorithm (g_{rl}) consistently outperforms SABRE (g_{sabre}) across all tested topologies.

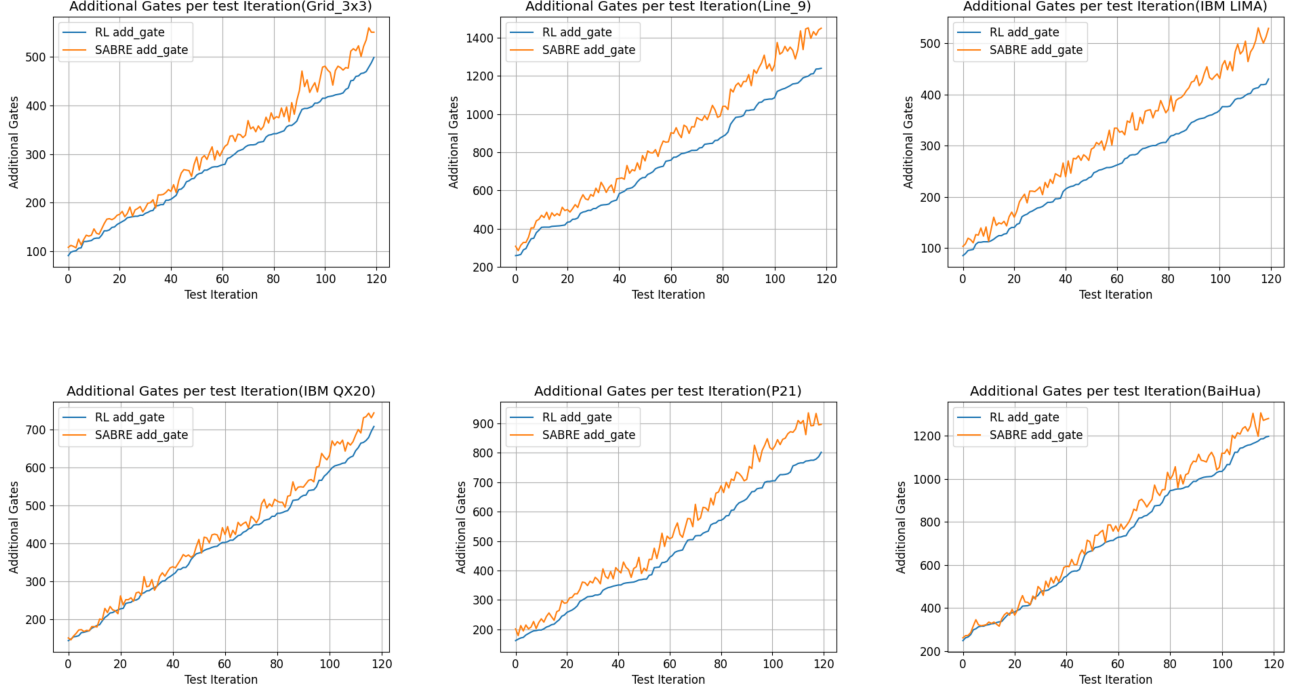


Fig. 7: Comparison of the proposed algorithm (denoted as g_{rl}) and SABRE (denoted as g_{sabre}) across several hardware topologies. The results demonstrate the superior performance of g_{rl} in minimizing SWAP gate insertions.

Specifically, g_{rl} achieves performance improvements ranging from 5.83% (IBM QX20) to 17.57% (IBM LIMA). Notably, in the IBM LIMA topology, g_{rl} reduces the number of SWAP gates from 37,748 to 31,113, a 17.57% improvement. Similarly, in the Line_9 and P21 topologies, g_{rl} achieves improvements of 13.35% and 13.33%, respectively. These results highlight the effectiveness of the proposed algorithm in minimizing SWAP gate insertions, which is critical for reducing circuit depth and improving quantum computation efficiency.

The superior performance of g_{rl} can be attributed to its reinforcement learning-based adaptive capability, which dynamically adjusts mapping strategies based on specific hardware topologies and quantum circuits. In contrast, SABRE, as a classical heuristic method, may lack the flexibility to fully exploit the structural characteristics of complex topologies. For instance, in the IBM LIMA topology, g_{rl} leverages the hardware's connectivity patterns to optimize gate placement, resulting in a significant reduction in SWAP gates. Similarly, in the Line_9 topology, which represents a linear arrangement of qubits, g_{rl} effectively minimizes long-range interactions by strategically inserting SWAP gates.

The reduction in SWAP gates is particularly important for quantum computing, as each SWAP gate increases circuit depth and introduces additional noise, thereby degrading the overall performance of quantum algorithms. By minimizing the number of SWAP gates, g_{rl} not only improves the execution efficiency of quantum circuits but also enhances the fidelity of quantum computations. This advantage is especially pronounced in large-scale circuits and complex hardware topologies, where the impact of SWAP gates is magnified.

In summary, the experimental results validate the effectiveness of the proposed algorithm in addressing the qubit mapping problem.

The consistent performance improvements across various topologies demonstrate the potential of reinforcement learning-based approaches for optimizing quantum circuit compilation. Future work will focus on extending the algorithm to support more diverse hardware architectures and further reducing the computational overhead of the mapping process.

C. Ablation Study

Our reinforcement learning framework introduces two key modifications to the original DQN algorithm: (1) an enhanced exploration strategy that incorporates topological information, and (2) a forward-looking reward mechanism. To evaluate the impact of these modifications, we conduct ablation studies on both components.

1) *Exploration Strategy*: As described in Section III-D, our exploration strategy combines random exploration with biased sampling based on topological information. This approach injects manually designed scoring information into the experience pool, guiding the agent and accelerating training. To validate its effectiveness, we compare it with a baseline strategy that relies solely on random exploration.

We train two agents on the IBM LIMA topology using these strategies and evaluate their performance on a pre-generated set of random circuits. As shown in Figure 8, the agent with biased sampling converges faster and achieves more stable performance compared to the random sampling baseline. Notably, both strategies ultimately achieve similar final performance, highlighting the importance of biased exploration in accelerating training.

2) *Forward-Looking Rewards*: In addition to immediate rewards for enabling two-qubit gates, our reward design incorporates a forward-looking term to encourage actions that benefit future mappings. To evaluate this mechanism, we compare two models: one

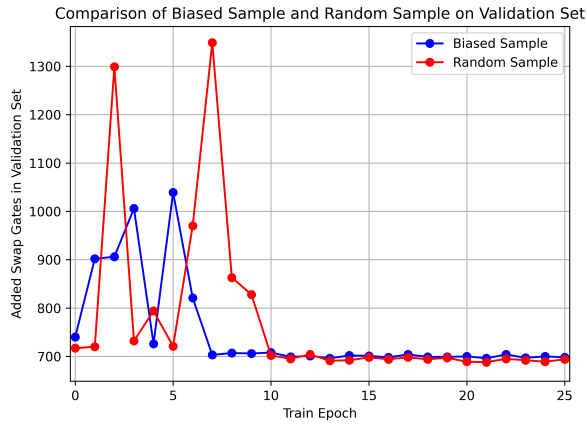


Fig. 8: Comparison of swap gates introduced by biased (blue) and random (red) sampling strategies on the validation set. The biased strategy converges in 7 epochs, while the random strategy requires 10 epochs. Each epoch takes approximately 45 minutes under the experimental configuration (The circuit segment size M_g is 500, the number of qubits n is 5 and the reward scaling factor C_R is 10).

with the basic reward structure (denoted as RL basic) and another with the forward-looking reward (denoted as RL look-ahead). Both models are trained on the Grid 3×3 topology and evaluated on a test set of 80 random circuits, each containing 100 to 500 two-qubit gates.

The RL look-ahead model introduces 13,406 SWAP gates, compared to 14,395 for the RL basic model, representing a 6.87% reduction. Figure 9 illustrates the performance comparison, demonstrating the clear advantage of the forward-looking reward mechanism in minimizing SWAP gate insertions.

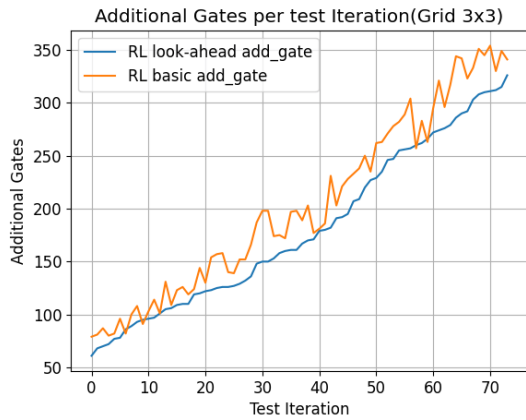


Fig. 9: Comparison of SWAP gates introduced by RL basic and RL look-ahead models on the Grid 3×3 topology. The circuit segment size M_g is 500, the number of qubits n is 9, the penalty term is set to -0.22, the reward scaling factor C_R is 10 and the look-ahead factor C_M is 2.

D. Extension Experiment

In Section III-E, we introduced an expansion algorithm to address the topology dependency of our trained models. While our reinforcement learning model is tailored to specific hardware topologies, the

expansion algorithm enables its application to larger-scale topologies by decomposing them into smaller, trained sub-topologies. For instance, a 5×5 grid can be viewed as a combination of multiple 3×3 grids, allowing the mapping process to be divided into smaller, manageable tasks.

To evaluate the effectiveness of this approach, we tested the expansion algorithm on a 5×5 grid (25 qubits) using a model trained on a 3×3 grid topology. We conducted 120 test rounds, each involving a randomly generated quantum circuit with 100 to 500 two-qubit gates. The results, shown in Figure 10, demonstrate the successful application of the expansion algorithm to larger topologies.

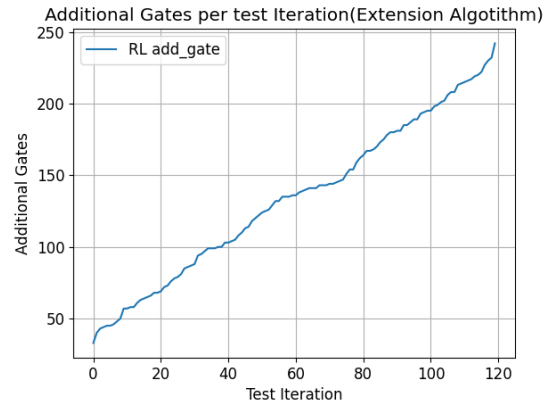


Fig. 10: Performance of the expansion algorithm on a 5×5 grid topology.

V. SUMMARY AND OUTLOOK

The quantum circuit mapping problem, which addresses the mismatch between quantum circuits and hardware constraints, is a critical challenge for practical quantum computing. This paper proposes a novel solution based on graph learning and reinforcement learning. Our approach introduces a graph-based deep reinforcement learning framework that encodes quantum circuits as directed acyclic graphs (DAGs) and leverages graph neural networks (GNNs) to extract structural features. The framework incorporates look-ahead rewards and biased exploration to guide optimal SWAP gate insertion, improving mapping efficiency and scalability.

Key contributions of our approach include:

- **Reduced SWAP Gates:** By utilizing GNNs to capture global circuit information, our algorithm reduces SWAP gates versus existing methods. Experimental results show 5.8% to 17.6% improvement over SABRE on large-scale circuits.
- **Interpretability:** The design is highly interpretable, with neural network outputs aligning with experimental data. Control experiments validate key design choices like heuristic-guided exploration and forward-looking rewards.

Despite these advancements, several areas for future improvement remain:

- 1) **Circuit Depth Optimization:** While minimizing SWAP gates, the algorithm does not explicitly optimize circuit depth. Future work could integrate depth-aware strategies to mitigate decoherence in the NISQ era.
- 2) **Generalized Topology Handling:** A unified model for multiple topologies would enhance scalability, addressing variations in decision spaces and reward scales across hardware.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *The 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 124–134.
- [2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *The 28th Annual ACM Symposium on Theory of Computing*, 1996, pp. 212–219.
- [3] J. Tilly, H. Chen, S. Cao, and et al., "The variational quantum eigensolver: a review of methods and best practices," *Physics Reports*, vol. 986, pp. 1–128, 2022.
- [4] J. Choi and J. Kim, "A tutorial on quantum approximate optimization algorithm (qaoa): Fundamentals and applications," in *2019 International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2019, pp. 138–142.
- [5] S. Lloyd, M. Mohseni, and P. Rebentrost, "Quantum principal component analysis," *Nature Physics*, vol. 10, no. 9, pp. 631–633, 2014.
- [6] P. Rebentrost, M. Mohseni, and S. Lloyd, "Quantum support vector machine for big data classification," *Physical Review Letters*, vol. 113, no. 13, p. 130503, 2014.
- [7] P. Q. Le, F. Dong, and K. Hirota, "A flexible representation of quantum images for polynomial preparation, image compression, and processing operations," *Quantum Information Processing*, vol. 10, pp. 63–84, 2011.
- [8] S. Ganguly, "Classification of nqr processed classical images using quantum neural networks (qnn)," *arXiv preprint arXiv:2204.02797*, 2022.
- [9] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [10] H. Huang, X. Xu, C. Guo, G. Tian, S. Wei, X. Sun, W. Bao, and G. Long, "Near-term quantum computing techniques: Variational quantum algorithms, error mitigation, circuit compilation, benchmarking and classical simulation," *CoRR*, vol. abs/2211.08737, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.08737>
- [11] A. A. DeAlmeida, G. W. Dueck, and A. C. DaSilva, "Finding optimal qubit permutations for ibm's quantum computer architectures," in *The 32nd Symposium on Integrated Circuits and Systems Design*. Association for Computing Machinery, 2019, pp. 1–6.
- [12] H. Deng, Y. Zhang, and Q. Li, "Codar: A contextual duration-aware qubit mapping for various nisq devices," in *The 57th ACM/IEEE Design Automation Conference*. IEEE, 2020, pp. 1–6.
- [13] G. Nannicini, L. S. Bishop, G. Ö et al., "Optimal qubit assignment and routing via integer programming," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, pp. 1–31, 2022.
- [14] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *The 24th International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 2019, pp. 1001–1014.
- [15] X. Zhou, Y. Feng, and S. Li, "A monte carlo tree search framework for quantum circuit transformation," in *The 39th International Conference on Computer-Aided Design*, 2020, pp. 1–7.
- [16] A. Botea, A. Kishimoto, and R. Marinescu, "On the complexity of quantum circuit compilation," in *Proceedings of the International Symposium on Combinatorial Search, Volume 9*, 2018, pp. 138–142.
- [17] M. Y. Siraichi, V. F. d. Santos, C. Collange et al., "Qubit allocation," in *The 16th International Symposium on Code Generation and Optimization*, 2018, pp. 113–125.
- [18] A. Shafaei, M. Saeedi, and M. Pedram, "Optimization of quantum circuits for interaction distance in linear nearest neighbor architectures," in *The 50th Annual Design Automation Conference*, 2013, pp. 1–6.
- [19] —, "Qubit placement to minimize communication overhead in 2d quantum architectures," in *2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2014, pp. 495–500.
- [20] A. Lye, R. Wille, and R. Drechsler, "Determining the minimal number of swap gates for multi-dimensional nearest neighbor quantum circuits," in *The 20th Asia and South Pacific Design Automation Conference*. IEEE, 2015, pp. 178–183.
- [21] D. Bhattacharjee and A. Chattopadhyay, "Depth-optimal quantum circuit placement for arbitrary topologies," *arXiv preprint arXiv:1703.08540*, 2017.
- [22] A. Molavi, A. Xu, M. Diges, and et al., "Qubit mapping and routing via maxsat," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1078–1091.
- [23] Y. Hirata, M. Nakanishi, S. Yamashita, and et al., "An efficient method to convert arbitrary quantum circuits to ones on a linear nearest neighbor architecture," in *2009 Third International Conference on Quantum, Nano and Micro Technologies*. IEEE, 2009, pp. 26–33.
- [24] M. Saeedi, R. Wille, and R. Drechsler, "Synthesis of quantum circuits for linear nearest neighbor architectures," *Quantum Information Processing*, vol. 10, pp. 355–377, 2011.
- [25] R. Wille, O. Keszocze, M. Walter, and et al., "Look-ahead schemes for nearest neighbor optimization of 1d and 2d quantum circuits," in *2016 21st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2016, pp. 292–297.
- [26] A. Zulehner, A. Paller, and R. Wille, "An efficient methodology for mapping quantum circuits to the ibm qx architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 7, pp. 1226–1236, 2018.
- [27] V. Saravanan and S. M. Saeed, "Noise adaptive quantum circuit mapping using reinforcement learning and graph neural network," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1374–1386, May 2023.
- [28] M. G. Pozzi, S. J. Herbert, A. Sengupta, and R. D. Mullins, "Using reinforcement learning to perform qubit routing in quantum compilers," *arXiv preprint arXiv:2007.15957*, 2020.
- [29] F. Scarselli, M. Gori, A. C. Tsoi, and et al., "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2008.
- [30] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [31] C. J. C. H. Watkins, "Learning from delayed rewards," 1989.
- [32] V. Mnih, K. Kavukcuoglu, D. Silver, and et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [33] T. Cai, S. Luo, K. Xu et al., "Graphnorm: A principled approach to accelerating graph neural network training," in *Proceedings of the International Conference on Machine Learning*, ser. PMLR, 2021, pp. 1204–1215.
- [34] G. Li, C. Xiong, A. Thabet et al., "Deepergcn: All you need to train deeper gcns," *arXiv preprint arXiv:2006.07739*, 2020.